

pandas 다중 인덱스(multi index)

```
In [1]: # 필요 모듈
import numpy as np
import pandas as pd
print(f'Numpy:{np.__version__}, Pandas:{pd.__version__}')
```

Numpy:2.5.2, Pandas:3.0.5

```
In [2]: # 여러 변수 출력 코드
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
```

```
In [3]: # 경고 메시지 무시
import warnings
warnings.filterwarnings('ignore')
```

참고. 난수 생성 : np.random 모듈

np.random.seed(seed값)

- seed : 난수 알고리즘에서 사용하는 기본 값
 - seed 값이 같으면 동일한 난수 발생
 - 예. np.random.seed(10)
- 계속 변경되는 난수를 생성하려면 시드값이 매번 변하도록 지정
 - 예. np.random.seed(int(time.time()))

난수 생성 함수

- np.random.rand() : 주어진 형태의 난수 배열 생성
- np.random.randint(최소값, 최대값, size=n)
 - [최소값, 최대값)의 범위에서 임의의 정수 생성
- np.random.randn() : 표준정규분포(Standard normal distribution)로부터 샘플링된 난수 생성
- np.random.standard_normal() : 표준정규분포 난수 발생
- np.random.normal([loc, scale, size]) : 정규분포 난수 생성
- np.random.random_sample(size) : [0,1)사이의 난수 생성
- np.random.choice(a[, size, replace, p]) : 주어진 배열로 부터 표본추출

```
In [4]: # 시드(seed) 값 고정
np.random.seed(10)
np.random.randint(1,7, size=10)
```

Out[4]: array([2, 6, 5, 1, 2, 4, 5, 2, 6, 1], dtype=int32)

```
In [7]: # 시드값 변경
import time

print(time.time())
np.random.seed(int(time.time()))
np.random.randint(1,7, size=10)
```

1788239687.8504753

Out[7]: array([6, 5, 4, 4, 6, 4, 2, 2, 3, 3], dtype=int32)

```
In [10]: np.random.rand() # 0~1 사이 실수난수 (균일분포 난수)
np.random.randn() # 표준정규분포 난수 (평균0, 표준편차1)
```

```
np.random.choice(['r','g','b','y','d'], size=2)
```

```
Out[10]: 0.3320493157598706
```

```
Out[10]: 0.6395366996114158
```

```
Out[10]: array(['g', 'b'], dtype='<U1')
```

참고. 파이썬의 random 모듈

<https://docs.python.org/ko/3/library/random.html>

정수 난수 발생 함수

`random.randrange(start, stop[, step])`

- `range(start, stop, step)`에서 임의로 선택된 요소를 반환
- `choice(range(start, stop, step))`와 동등하지만 실제로 `range` 객체를 만들지는 않음

`random.randint(a, b)`

- `a <= N <= b` 를 만족하는 임의의 정수 N을 반환
- `randrange(a, b+1)`의 별칭

시퀀스 난수 발생 함수

`random.choice(seq)`

- 비어 있지 않은 시퀀스 `seq`에서 임의의 요소를 반환
- `seq`가 비어 있으면, `IndexError`를 발생

`random.choices(population, weights=None, *, cum_weights=None, k=1)`

- `population`에서 중복을 허락하면서(with replacement) 선택한 `k` 크기의 요소 리스트를 반환
- `population`이 비어 있으면 `IndexError` 발생
- `weights` 시퀀스가 지정되면 상대 가중치에 따라 선택됨
- `weights`나 `cum_weights`를 지정하지 않으면 같은 확률로 선택
- `weights` 시퀀스가 제공되면, `population` 시퀀스와 길이가 같아야 함
- `weights`와 `cum_weights`를 모두 지정하는 것은 `TypeError`

`random.sample(population, k, *, counts=None)`

- `population` 시퀀스로부터 추출한 `k`개 길이의 새 리스트를 반환
- random sampling without replacement

실수 난수 발생 함수

`random.random()`

- `0.0 <= X < 1.0` 사이의 실수 반환

`random.uniform(a, b)`

- `a <= b` 일 때 `a <= N <= b`, `b < a` 일 때 `b <= N <= a` 를 만족하는 임의의 부동 소수점 숫자 N을 반환
 - 종단 값 `b`는 방정식 `a + (b-a) * random()`의 부동 소수점 자리 올림에 따라 범위에 포함되거나 포함되지 않을 수 있음
-

다중 인덱스(multi index)란?

- 행이나 열 인덱스가 계층으로 구성된 인덱스(Hierarchical indexing)

```
In [11]: df = pd.DataFrame([[1,2,3,4],[2,3,4,5]],
                           columns=list('ABCD'),
                           index=[['a','b']])

df
df.index
```

```
Out[11]:
```

	A	B	C	D
a	1	2	3	4
b	2	3	4	5

```
Out[11]: MultiIndex([('a',),
                     ('b',)],
                    )
```

[학습 내용]

1. 다중 인덱스를 갖는 Series
2. 다중 인덱스를 갖는 DataFrame
3. MultiIndex 객체
4. 다중인덱스의 특정 레벨 제거 : droplevel()
5. 행인덱스 레벨 해제 : unstack()
6. 열인덱스 레벨 해제 : stack()
7. 다중인덱스의 레벨 교환 : swaplevel()
8. 다중인덱스의 행/열 추가
9. 다중인덱스 정렬

1. 다중인덱스를 갖는 Series

예1. 난수 데이터를 갖는 Series

```
In [12]: arrays = [np.array(["bar", "bar", "baz", "baz",
                             "foo", "foo", "qux", "qux"]),
                  np.array(["one", "two", "one", "two",
                             "one", "two", "one", "two"])]

arrays
s1 = pd.Series(np.random.randn(8), index=arrays)
s1
```

```
Out[12]: [array(['bar', 'bar', 'baz', 'baz', 'foo', 'foo', 'qux', 'qux'],
              dtype='<U3'),
          array(['one', 'two', 'one', 'two', 'one', 'two', 'one', 'two'],
              dtype='<U3')]
```

```
Out[12]: bar one    0.375315
         two    1.307325
         baz one   -0.061750
         two    0.464888
         foo one    1.319441
         two   -0.142737
         qux one   -0.100496
         two   -0.069586
dtype: float64
```

예2. 키를 튜플로 갖는 딕셔너리의 데이터로 Series 생성

```
In [13]: # 키를 튜플로 갖는 딕셔너리 데이터
data = {('James', 'Eng'): 100,
        ('James', 'Math') : 90,
        ('Ted', 'Eng') : 90,
        ('Ted', 'Math') : 70,
        ('Adam', 'Eng') : 85,
        ('Adam', 'Math') : 90 }

data
s2 = pd.Series(data)
s2
```

```
Out[13]: {('James', 'Eng'): 100,
          ('James', 'Math'): 90,
          ('Ted', 'Eng'): 90,
          ('Ted', 'Math'): 70,
          ('Adam', 'Eng'): 85,
          ('Adam', 'Math'): 90}
```

```
Out[13]: James  Eng      100
          Math      90
          Ted   Eng      90
          Math      70
          Adam  Eng      85
          Math      90
dtype: int64
```

```
In [14]: s2.index
```

```
Out[14]: MultiIndex([('James', 'Eng'),
                    ('James', 'Math'),
                    ('Ted', 'Eng'),
                    ('Ted', 'Math'),
                    ('Adam', 'Eng'),
                    ('Adam', 'Math')],
                    )
```

인덱스의 이름 지정 : 시리즈.index.names = [,]

```
In [18]: s2.index.names
s2.index.names = ['name', 'course']
s2
```

```
Out[18]: FrozenList([None, None])
```

```
Out[18]: name  course
James  Eng      100
        Math      90
Ted    Eng      90
        Math      70
Adam   Eng      85
        Math      90
dtype: int64
```

다중인덱스를 갖는 Series의 인덱싱

- 시리즈[상위인덱스]
- 시리즈.상위인덱스
- 시리즈[(상위인덱스, 하위인덱스)]
- 시리즈.상위인덱스.하위인덱스
- 시리즈[:, 하위인덱스]

```
In [20]: s2['James']  
s2.James
```

```
Out[20]: course  
Eng      100  
Math      90  
dtype: int64
```

```
Out[20]: course  
Eng      100  
Math      90  
dtype: int64
```

```
In [27]: s2[('Ted', 'Math')]  
s2.Ted.Eng  
s2['James', 'Math']
```

```
Out[27]: np.int64(70)
```

```
Out[27]: np.int64(90)
```

```
Out[27]: np.int64(90)
```

```
In [25]: s2[['James', 'Adam']]
```

```
Out[25]: name  course  
James  Eng      100  
        Math      90  
Adam   Eng      85  
        Math      90  
dtype: int64
```

```
In [28]: s2[:, 'Math']
```

```
Out[28]: name  
James      90  
Ted         70  
Adam        90  
dtype: int64
```

2. 다중 인덱스를 갖는 DataFrame

- 데이터 프레임 생성 시 생성자에서 columns인수나 index 인수를 2차원 리스트(행렬) 형태로 지정할 경우

1) column인덱스를 다중 인덱스로 갖는 DataFrame

```
In [29]: np.random.seed(0)  
data = np.round(np.random.randn(5,4), 2)  
data
```

```
Out[29]: array([[ 1.76,  0.4 ,  0.98,  2.24],  
                [ 1.87, -0.98,  0.95, -0.15],  
                [-0.1 ,  0.41,  0.14,  1.45],  
                [ 0.76,  0.12,  0.44,  0.33],  
                [ 1.49, -0.21,  0.31, -0.85]])
```

```
In [31]: df = pd.DataFrame(data=data,  
                           columns=[['A', 'A', 'B', 'B'], ['C1', 'C2', 'C3', 'C4']])  
df  
df.columns
```

```
Out[31]:
```

	A			B
	C1	C2	C3	C4
0	1.76	0.40	0.98	2.24
1	1.87	-0.98	0.95	-0.15
2	-0.10	0.41	0.14	1.45
3	0.76	0.12	0.44	0.33
4	1.49	-0.21	0.31	-0.85

```
Out[31]: MultiIndex([('A', 'C1'),
                    ('A', 'C2'),
                    ('B', 'C3'),
                    ('B', 'C4')],
                    )
```

열 인덱싱(1): df[상위인덱스]

- 상위 인덱스의 모든 열에 대한 데이터프레임 반환

```
In [32]: df['A']
```

```
Out[32]:
```

	C1	C2
0	1.76	0.40
1	1.87	-0.98
2	-0.10	0.41
3	0.76	0.12
4	1.49	-0.21

열 인덱싱(2): df[(상위인덱스, 하위인덱스)]

- 인덱스가 하나가 아니므로 묶어서(튜플로) 전달해야 함
- 시리즈로 반환

```
In [33]: df[('A', 'C1')]
```

```
Out[33]: 0    1.76
1    1.87
2   -0.10
3    0.76
4    1.49
Name: (A, C1), dtype: float64
```

```
In [34]: df['A', 'C1']
```

```
Out[34]: 0    1.76
1    1.87
2   -0.10
3    0.76
4    1.49
Name: (A, C1), dtype: float64
```

열인덱싱(3) : . 연산자로 확장

- df.상위인덱스

- df.상위인덱스.하위인덱스

```
In [35]: df.A
df.A.C2
```

```
Out[35]:
```

	C1	C2
0	1.76	0.40
1	1.87	-0.98
2	-0.10	0.41
3	0.76	0.12
4	1.49	-0.21

```
Out[35]: 0    0.40
1   -0.98
2    0.41
3    0.12
4   -0.21
Name: C2, dtype: float64
```

열인덱스 이름 지정 : df.columns.names = []

```
In [37]: df.columns
df.columns.names
df.columns.names = ['upper', 'lower']
df
```

```
Out[37]: MultiIndex([('A', 'C1'),
                    ('A', 'C2'),
                    ('B', 'C3'),
                    ('B', 'C4')],
                    )
```

```
Out[37]: FrozenList([None, None])
```

```
Out[37]:
```

	upper	A		B	
	lower	C1	C2	C3	C4
	0	1.76	0.40	0.98	2.24
	1	1.87	-0.98	0.95	-0.15
	2	-0.10	0.41	0.14	1.45
	3	0.76	0.12	0.44	0.33
	4	1.49	-0.21	0.31	-0.85

2) 행인덱스를 다중 인덱스로 갖는 DataFrame

```
In [38]: data2 = np.random.randint(1,10, size=(4,4))
data2
df2 = pd.DataFrame(data2,
                    index=[['a','a','b','b'],
                          ['1','2','1','2']],
                    columns= ['A','B','C','D'])

df2
df2.index
```

```
Out[38]: array([[1, 5, 8, 4],
                [3, 8, 3, 1],
                [1, 5, 6, 6],
                [7, 9, 5, 2]], dtype=int32)
```

```
Out[38]:
```

	A	B	C	D	
a	1	1	5	8	4
	2	3	8	3	1
b	1	1	5	6	6
	2	7	9	5	2

```
Out[38]: MultiIndex([(a, 1),
                    (a, 2),
                    (b, 1),
                    (b, 2)],
                    )
```

행인덱싱(1) : df.loc[상위인덱스]

```
In [39]: df2.loc['a']
df2.loc['a':'b']
```

```
Out[39]:
```

	A	B	C	D
1	1	5	8	4
2	3	8	3	1

```
Out[39]:
```

	A	B	C	D	
a	1	1	5	8	4
	2	3	8	3	1
b	1	1	5	6	6
	2	7	9	5	2

행인덱싱(2) : df.loc[(상위인덱스, 하위인덱스)]

- 상위인덱스와 하위인덱스를 튜플로 전달

```
In [41]: df2.loc[('a', '1')]
df2.loc['a', '1']
```

```
Out[41]: A    1
         B    5
         C    8
         D    4
         Name: (a, 1), dtype: int32
```

```
Out[41]: A    1
         B    5
         C    8
         D    4
         Name: (a, 1), dtype: int32
```

인덱서.iloc : df.iloc[]

- .iloc인덱서는 행이름, 열이름 기반이 아님


```
In [42]: df2
df2.iloc[0]
```

```
Out[42]:
```

	A	B	C	D	
a	1	1	5	8	4
	2	3	8	3	1
b	1	1	5	6	6
	2	7	9	5	2

```
Out[42]: A    1
B    5
C    8
D    4
Name: (a, 1), dtype: int32
```

```
In [43]: df2.iloc[1, 1:]
```

```
Out[43]: B    8
C    3
D    1
Name: (a, 2), dtype: int32
```

```
In [44]: df2.iloc[0,0]
```

```
Out[44]: np.int32(1)
```

3) 행과 열에 모두 다중인덱스를 갖는 DataFrame

```
In [45]: np.random.seed(0)
data3 = np.round(np.random.randn(6,4),2)
col1 = ['A']*2 + ['B']*2
col2 = ['C'+str(i) for i in range(1,5)]
columns = [col1, col2]
columns
idx1 = ['M']*3 + ['F']*3
idx2 = ['id_'+str(i) for i in range(1,4)]*2
index = [idx1, idx2]
index
```

```
Out[45]: [['A', 'A', 'B', 'B'], ['C1', 'C2', 'C3', 'C4']]
```

```
Out[45]: [['M', 'M', 'M', 'F', 'F', 'F'],
          ['id_1', 'id_2', 'id_3', 'id_1', 'id_2', 'id_3']]
```

```
In [46]: df3 = pd.DataFrame(data3, index=index, columns=columns)
df3
```

Out[46]:

		A		B	
		C1	C2	C3	C4
M	id_1	1.76	0.40	0.98	2.24
	id_2	1.87	-0.98	0.95	-0.15
	id_3	-0.10	0.41	0.14	1.45
F	id_1	0.76	0.12	0.44	0.33
	id_2	1.49	-0.21	0.31	-0.85
	id_3	-2.55	0.65	0.86	-0.74

행/열 각 인덱스에 이름(names) 설정

- 이름을 지정하면 직관성이 높아지고 편리하게 사용할 수 있음
- 열이름/행이름 구분하는데 용이
- 문법
 - df.columns.names = 값 또는 리스트
 - df.index.names = 값 또는 리스트

```
In [49]: df3.columns.names = ['Cidx1', 'Cidx2']
df3.index.names = ['Ridx1', 'Ridx2']
df3
```

Out[49]:

		Cidx1		A		B	
		Cidx2	C1	C2	C3	C4	
Ridx1	Ridx2						
M	id_1	1.76	0.40	0.98	2.24		
	id_2	1.87	-0.98	0.95	-0.15		
	id_3	-0.10	0.41	0.14	1.45		
F	id_1	0.76	0.12	0.44	0.33		
	id_2	1.49	-0.21	0.31	-0.85		
	id_3	-2.55	0.65	0.86	-0.74		

```
In [50]: df3.index
df3.columns
```

```
Out[50]: MultiIndex([(M, id_1),
                      (M, id_2),
                      (M, id_3),
                      (F, id_1),
                      (F, id_2),
                      (F, id_3)],
                    names=['Ridx1', 'Ridx2'])
```

```
Out[50]: MultiIndex([(A, C1),
                      (A, C2),
                      (B, C3),
                      (B, C4)],
                    names=['Cidx1', 'Cidx2'])
```

3. MultiIndex 객체

- https://pandas.pydata.org/docs/user_guide/advanced.html

- 생성 방법

1. MultiIndex.from_arrays() 사용 : 배열(array)의 리스트
2. MultiIndex.from_tuples() : 튜플들(tuples)의 리스트
3. MultiIndex.from_product() : 리스트의 cross product
4. MultiIndex.from_frame() : DataFrame

In [51]: *# 1. MultiIndex.from_arrays() 이용한 다중인덱스 생성*

```
arrays = np.array([["one", "two", "one", "two"],
                  ["bar", "baz", "foo", "qux"]])
arrays

index = pd.MultiIndex.from_arrays(arrays,
                                  names=["first", "second"])
index
```

Out[51]: array([['one', 'two', 'one', 'two'],
 ['bar', 'baz', 'foo', 'qux']], dtype='<U3')

Out[51]: MultiIndex([('one', 'bar'),
 ('two', 'baz'),
 ('one', 'foo'),
 ('two', 'qux')],
 names=['first', 'second'])

In [52]: *# 2. MultiIndex.from_tuples() 이용한 다중인덱스 생성*

```
arrays = [
    ["bar", "bar", "baz", "baz", "foo", "foo", "qux", "qux"],
    ["one", "two", "one", "two", "one", "two", "one", "two"]]

tuples = list(zip(*arrays))
tuples

index = pd.MultiIndex.from_tuples(tuples,
                                  names=["first", "second"])
index
```

Out[52]: [('bar', 'one'),
 ('bar', 'two'),
 ('baz', 'one'),
 ('baz', 'two'),
 ('foo', 'one'),
 ('foo', 'two'),
 ('qux', 'one'),
 ('qux', 'two')]

Out[52]: MultiIndex([('bar', 'one'),
 ('bar', 'two'),
 ('baz', 'one'),
 ('baz', 'two'),
 ('foo', 'one'),
 ('foo', 'two'),
 ('qux', 'one'),
 ('qux', 'two')],
 names=['first', 'second'])

In [53]: *# 3. MultiIndex.from_product() 사용한 다중인덱스 생성*

```
iterables = [ ["bar", "baz", "foo", "qux"], ["one", "two"] ]
pd.MultiIndex.from_product(iterables,
                           names=["first", "second"])
```

```
Out[53]: MultiIndex([('bar', 'one'),
                    ('bar', 'two'),
                    ('baz', 'one'),
                    ('baz', 'two'),
                    ('foo', 'one'),
                    ('foo', 'two'),
                    ('qux', 'one'),
                    ('qux', 'two')],
                    names=['first', 'second'])
```

In [54]: *# 4. MultiIndex.from_frame()를 사용한 다중인덱스 생성*

```
df = pd.DataFrame([["bar", "one"], ["bar", "two"],
                  ["foo", "one"], ["foo", "two"]],
                  columns=["first", "second"])

df
pd.MultiIndex.from_frame(df)
```

```
Out[54]:
```

	first	second
0	bar	one
1	bar	two
2	foo	one
3	foo	two

```
Out[54]: MultiIndex([('bar', 'one'),
                    ('bar', 'two'),
                    ('foo', 'one'),
                    ('foo', 'two')],
                    names=['first', 'second'])
```

예제. MultiIndex객체 생성하여 다중인덱스 설정

```
In [55]: data4 = np.round(np.random.randn(4,9), 1)
data4

index = pd.MultiIndex.from_product([["1995", "2000"], ["May", "Dec"]], names=["Year", "Month"])
index

columns = pd.MultiIndex.from_product([["A", "B", "C"], [1,2,3]], names=["name", "count"])
columns

df4 = pd.DataFrame(data=data4, index=index, columns=columns)
df4
```

```
Out[55]: array([[ 2.3, -1.5,  0. , -0.2,  1.5,  1.5,  0.2,  0.4, -0.9],
                [-2. , -0.3,  0.2,  1.2,  1.2, -0.4, -0.3, -1. , -1.4],
                [-1.7,  2. , -0.5, -0.4, -1.3,  0.8, -1.6, -0.2, -0.9],
                [ 0.4, -0.5, -1.2, -0. ,  0.4,  0.1,  0.3, -0.6, -0.4]])
```

```
Out[55]: MultiIndex([(1995, 'May'),
                    (1995, 'Dec'),
                    (2000, 'May'),
                    (2000, 'Dec')],
                    names=['Year', 'Month'])
```

```
Out[55]: MultiIndex([( 'A', 1),
                    ( 'A', 2),
                    ( 'A', 3),
                    ( 'B', 1),
                    ( 'B', 2),
                    ( 'B', 3),
                    ( 'C', 1),
                    ( 'C', 2),
                    ( 'C', 3)],
                    names=[ 'name', 'count'])
```

Out[55]:

name		A			B			C		
count		1	2	3	1	2	3	1	2	3
Year	Month									
1995	May	2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9
	Dec	-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4
2000	May	-1.7	2.0	-0.5	-0.4	-1.3	0.8	-1.6	-0.2	-0.9
	Dec	0.4	-0.5	-1.2	-0.0	0.4	0.1	0.3	-0.6	-0.4

```
In [59]: df4['A']
df4.loc[1995]
```

Out[59]:

		count	1	2	3
	Year	Month			
	1995	May	2.3	-1.5	0.0
		Dec	-2.0	-0.3	0.2
	2000	May	-1.7	2.0	-0.5
		Dec	0.4	-0.5	-1.2

Out[59]:

name		A			B			C		
count		1	2	3	1	2	3	1	2	3
Month										
	May	2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9
	Dec	-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4

```
In [61]: df4
df4.reset_index()
```

Out[61]:

name		A			B			C		
count		1	2	3	1	2	3	1	2	3
Year	Month									
1995	May	2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9
	Dec	-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4
2000	May	-1.7	2.0	-0.5	-0.4	-1.3	0.8	-1.6	-0.2	-0.9
	Dec	0.4	-0.5	-1.2	-0.0	0.4	0.1	0.3	-0.6	-0.4

```
Out[61]:
```

	name	Year	Month	A			B			C		
	count			1	2	3	1	2	3	1	2	3
0	1995	May	2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9	
1	1995	Dec	-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4	
2	2000	May	-1.7	2.0	-0.5	-0.4	-1.3	0.8	-1.6	-0.2	-0.9	
3	2000	Dec	0.4	-0.5	-1.2	-0.0	0.4	0.1	0.3	-0.6	-0.4	

4. 다중인덱스의 특정 레벨 제거 : droplevel(level, axis)

1) 시리즈의 다중인덱스 레벨 제거

```
In [62]: s2
```

```
Out[62]:
```

name	course	
James	Eng	100
	Math	90
Ted	Eng	90
	Math	70
Adam	Eng	85
	Math	90

dtype: int64

```
In [63]: s2.droplevel(0)
```

```
Out[63]:
```

course	
Eng	100
Math	90
Eng	90
Math	70
Eng	85
Math	90

dtype: int64

```
In [64]: s2.droplevel(1)
```

```
Out[64]:
```

name	
James	100
James	90
Ted	90
Ted	70
Adam	85
Adam	90

dtype: int64

2) 데이터프레임에서 다중인덱스 레벨 제거

```
In [65]: df4
```

Out[65]:

name		A			B			C		
count		1	2	3	1	2	3	1	2	3
Year	Month									
1995	May	2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9
	Dec	-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4
2000	May	-1.7	2.0	-0.5	-0.4	-1.3	0.8	-1.6	-0.2	-0.9
	Dec	0.4	-0.5	-1.2	-0.0	0.4	0.1	0.3	-0.6	-0.4

행인덱스 레벨 제거

In [66]:

```
df4.droplevel(0)
```

Out[66]:

name		A			B			C		
count		1	2	3	1	2	3	1	2	3
Month										
May		2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9
Dec		-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4
May		-1.7	2.0	-0.5	-0.4	-1.3	0.8	-1.6	-0.2	-0.9
Dec		0.4	-0.5	-1.2	-0.0	0.4	0.1	0.3	-0.6	-0.4

In [67]:

```
df4.droplevel(1) # axis=0 기본값
```

Out[67]:

name		A			B			C		
count		1	2	3	1	2	3	1	2	3
Year										
1995	2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9	
	-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4	
2000	-1.7	2.0	-0.5	-0.4	-1.3	0.8	-1.6	-0.2	-0.9	
	0.4	-0.5	-1.2	-0.0	0.4	0.1	0.3	-0.6	-0.4	

열인덱스 레벨 제거

In [68]:

```
df4.droplevel(0, axis=1)
```

Out[68]:

		count	1	2	3	1	2	3	1	2	3
Year	Month										
1995	May	2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9	
	Dec	-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4	
2000	May	-1.7	2.0	-0.5	-0.4	-1.3	0.8	-1.6	-0.2	-0.9	
	Dec	0.4	-0.5	-1.2	-0.0	0.4	0.1	0.3	-0.6	-0.4	

```
In [69]: df4.droplevel(1, axis=1)
```

```
Out[69]:
```

	name	A	A	A	B	B	B	C	C	C
	Year	Month								
1995	May	2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9
	Dec	-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4
2000	May	-1.7	2.0	-0.5	-0.4	-1.3	0.8	-1.6	-0.2	-0.9
	Dec	0.4	-0.5	-1.2	-0.0	0.4	0.1	0.3	-0.6	-0.4

5. 행인덱스 레벨 해제 : unstack()

: 행인덱스 -> 열인덱스로 변환

1) 시리즈의 다중인덱스 레벨 해제

- 해제된 레벨 인덱스는 열인덱스로 변경되며, 데이터프레임으로 반환됨
- 형식

```
Series.unstack(level=-1, fill_value=None, sort=True)
```

- level : int, str, or list of these, default last level
 - Level(s) to unstack, can pass level name.
- fill_value : scalar value, default None
 - Value to use when replacing NaN values.
- sort : bool, default True
 - Sort the level(s) in the resulting MultiIndex columns.

<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.unstack.html>

```
In [71]: s1
```

```
Out[71]:
```

bar	one	0.375315
	two	1.307325
baz	one	-0.061750
	two	0.464888
foo	one	1.319441
	two	-0.142737
qux	one	-0.100496
	two	-0.069586

dtype: float64

마지막 레벨 해제 : unstack(level=-1)

```
In [72]: s1.unstack()  
s1.unstack(level=-1)
```

```
Out[72]:
```

	one	two
bar	0.375315	1.307325
baz	-0.061750	0.464888
foo	1.319441	-0.142737
qux	-0.100496	-0.069586

Out[72]:

	one	two
bar	0.375315	1.307325
baz	-0.061750	0.464888
foo	1.319441	-0.142737
qux	-0.100496	-0.069586

첫번째 레벨 해제 : `unstack(level=0)`

In [73]: `s1.unstack(level=0)`

Out[73]:

	bar	baz	foo	qux
one	0.375315	-0.061750	1.319441	-0.100496
two	1.307325	0.464888	-0.142737	-0.069586

2) 데이터프레임의 다중 행인덱스 레벨 해제

- 데이터프레임의 인덱스 레벨 해제
- 해제된 레벨은 열인덱스 중 가장 마지막 레벨이 됨

[형식]

`DataFrame.unstack(level=-1, fill_value=None)`

- `level` : int, str, or list of these, default= -1 (last level)
 - Level(s) of index to unstack, can pass level name.
- `fill_value` : int, str or dict
 - Replace NaN with this value if the unstack produces missing values.
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.unstack.html>

In [74]: `df4`

Out[74]:

name		A			B			C		
count		1	2	3	1	2	3	1	2	3
Year	Month									
1995	May	2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9
	Dec	-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4
2000	May	-1.7	2.0	-0.5	-0.4	-1.3	0.8	-1.6	-0.2	-0.9
	Dec	0.4	-0.5	-1.2	-0.0	0.4	0.1	0.3	-0.6	-0.4

In [76]: `df4.unstack()`

행인덱스의 마지막 레벨이 해제되면서, 컬럼인덱스의 마지막레벨로 들어감

Out[76]:

name		A						B									
count		1		2		3		1		2		3		1		2	
Month	Dec	May	Dec	May	Dec	May	Dec	May	Dec	May	Dec	May	Dec	May	Dec	May	Dec
Year																	
1995	-2.0	2.3	-0.3	-1.5	0.2	0.0	1.2	-0.2	1.2	1.5	-0.4	1.5	-0.3	0.2	-1.0	0.4	-
2000	0.4	-1.7	-0.5	2.0	-1.2	-0.5	-0.0	-0.4	0.4	-1.3	0.1	0.8	0.3	-1.6	-0.6	-0.2	-

In [77]: `df4.unstack(0)`

Out[77]:

name		A						B							
count		1		2		3		1		2		3		1	
Year	Month	1995	2000	1995	2000	1995	2000	1995	2000	1995	2000	1995	2000	1995	2000
	Dec	-2.0	0.4	-0.3	-0.5	0.2	-1.2	1.2	-0.0	1.2	0.4	-0.4	0.1	-0.3	0.3
	May	2.3	-1.7	-1.5	2.0	0.0	-0.5	-0.2	-0.4	1.5	-1.3	1.5	0.8	0.2	-1.6

In [79]: `# 레벨의 이름을 지정하여 해제`
`df4.unstack('Month')`
`# df4.unstack(level='Month')`

Out[79]:

name		A						B									
count		1		2		3		1		2		3		1		2	
Month	Year	Dec	May	Dec	May	Dec	May	Dec	May	Dec	May	Dec	May	Dec	May	Dec	May
1995		-2.0	2.3	-0.3	-1.5	0.2	0.0	1.2	-0.2	1.2	1.5	-0.4	1.5	-0.3	0.2	-1.0	0.4
2000		0.4	-1.7	-0.5	2.0	-1.2	-0.5	-0.0	-0.4	0.4	-1.3	0.1	0.8	0.3	-1.6	-0.6	-0.2

6. 열인덱스 레벨 해제 : stack()

: 열인덱스 -> 행인덱스로 변환

데이터프레임에서 열인덱스 레벨 해제

- 지정한 열인덱스가 행인덱스의 마지막 레벨로 변환 추가됨
- single level 열인덱스를 갖는 경우 시리즈로 반환
- multi level 열인덱스를 갖는 경우 데이터프레임 반환

[형식]

`DataFrame.stack(level= -1, dropna=True)`
 - level 위치 또는 열이름 지정
 - level : int, str, list, default= -1
 - dropna : bool, default True

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.stack.html>

예1. 1차 레벨을 가진 데이터프레임

```
In [80]: df = pd.DataFrame(data=[[0,1],[2,3]],
                           index=['cat','dog'],
                           columns=['weight','height'])
df
```

```
Out[80]:
```

	weight	height
cat	0	1
dog	2	3

```
In [81]: df.stack()
```

```
Out[81]: cat weight    0
          height    1
          dog weight    2
          height    3
dtype: int64
```

```
In [83]: df.stack(-1)
```

```
Out[83]: cat weight    0
          height    1
          dog weight    2
          height    3
dtype: int64
```

예2. 다중레벨을 갖는 데이터프레임

```
In [84]: df4
```

```
Out[84]:
```

	name			A			B			C
	count	1	2	3	1	2	3	1	2	3
Year	Month									
1995	May	2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9
	Dec	-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4
2000	May	-1.7	2.0	-0.5	-0.4	-1.3	0.8	-1.6	-0.2	-0.9
	Dec	0.4	-0.5	-1.2	-0.0	0.4	0.1	0.3	-0.6	-0.4

```
In [85]: df4.stack()
# 열의 마지막 레벨 인덱스가 행의 마지막 레벨로 들어감(쌓임)
```

Out[85]:

		name	A	B	C	
	Year	Month	count			
	1995	May	1	2.3	-0.2	0.2
			2	-1.5	1.5	0.4
			3	0.0	1.5	-0.9
		Dec	1	-2.0	1.2	-0.3
			2	-0.3	1.2	-1.0
			3	0.2	-0.4	-1.4
	2000	May	1	-1.7	-0.4	-1.6
			2	2.0	-1.3	-0.2
			3	-0.5	0.8	-0.9
		Dec	1	0.4	-0.0	0.3
			2	-0.5	0.4	-0.6
			3	-1.2	0.1	-0.4

In [86]:

df4.stack().stack()

```
Out[86]: Year  Month  count  name
1995   May    1      A      2.3
        May    1      B     -0.2
        May    1      C      0.2
        May    2      A     -1.5
        May    2      B      1.5
        May    2      C      0.4
        May    3      A      0.0
        May    3      B      1.5
        May    3      C     -0.9
        Dec    1      A     -2.0
        Dec    1      B      1.2
        Dec    1      C     -0.3
        Dec    2      A     -0.3
        Dec    2      B      1.2
        Dec    2      C     -1.0
        Dec    3      A      0.2
        Dec    3      B     -0.4
        Dec    3      C     -1.4
2000   May    1      A     -1.7
        May    1      B     -0.4
        May    1      C     -1.6
        May    2      A      2.0
        May    2      B     -1.3
        May    2      C     -0.2
        May    3      A     -0.5
        May    3      B      0.8
        May    3      C     -0.9
        Dec    1      A      0.4
        Dec    1      B     -0.0
        Dec    1      C      0.3
        Dec    2      A     -0.5
        Dec    2      B      0.4
        Dec    2      C     -0.6
        Dec    3      A     -1.2
        Dec    3      B      0.1
        Dec    3      C     -0.4

dtype: float64
```

```
In [87]: df4.stack(level=0)
```

Out[87]:

			count	1	2	3
Year	Month	name				
1995	May	A	2.3	-1.5	0.0	
		B	-0.2	1.5	1.5	
		C	0.2	0.4	-0.9	
	Dec	A	-2.0	-0.3	0.2	
		B	1.2	1.2	-0.4	
		C	-0.3	-1.0	-1.4	
	May	A	-1.7	2.0	-0.5	
		B	-0.4	-1.3	0.8	
		C	-1.6	-0.2	-0.9	
2000	Dec	A	0.4	-0.5	-1.2	
		B	-0.0	0.4	0.1	
		C	0.3	-0.6	-0.4	

예3. 열인덱스 이름을 갖는 데이터프레임

In [88]:

df4.columns.names

Out[88]: FrozenList(['name', 'count'])

In [89]:

df4.stack('name')

Out[89]:

			count	1	2	3
Year	Month	name				
1995	May	A	2.3	-1.5	0.0	
		B	-0.2	1.5	1.5	
		C	0.2	0.4	-0.9	
	Dec	A	-2.0	-0.3	0.2	
		B	1.2	1.2	-0.4	
		C	-0.3	-1.0	-1.4	
	May	A	-1.7	2.0	-0.5	
		B	-0.4	-1.3	0.8	
		C	-1.6	-0.2	-0.9	
2000	Dec	A	0.4	-0.5	-1.2	
		B	-0.0	0.4	0.1	
		C	0.3	-0.6	-0.4	

In [90]:

df4.stack('count')

Out[90]:

		name	A	B	C
Year	Month	count			
1995	May	1	2.3	-0.2	0.2
		2	-1.5	1.5	0.4
		3	0.0	1.5	-0.9
	Dec	1	-2.0	1.2	-0.3
		2	-0.3	1.2	-1.0
		3	0.2	-0.4	-1.4
2000	May	1	-1.7	-0.4	-1.6
		2	2.0	-1.3	-0.2
		3	-0.5	0.8	-0.9
	Dec	1	0.4	-0.0	0.3
		2	-0.5	0.4	-0.6
		3	-1.2	0.1	-0.4

7. 다중인덱스의 레벨 교환 : swaplevel()

[형식]

`DataFrame.swaplevel(i=-2, j=-1, axis=0)`

- `i, j` : int or str
 - Levels of the indices to be swapped. Can pass level name as string.
- `axis` : 0 or 'index', 1 or 'columns', default=0
 - The axis to swap levels on
 - 0 or 'index' for row-wise
 - 1 or 'columns' for column-wise
- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.swaplevel.html>

```
In [91]: df = pd.DataFrame({"Grade": ["A", "B", "A", "C"]},
                           index=[
                               ["Final exam", "Final exam", "Coursework", "Coursework"],
                               ["History", "Geography", "History", "Geography"],
                               ["January", "February", "March", "April"],])
df
```

Out[91]:

		Grade	
Final exam	History	January	A
	Geography	February	B
Coursework	History	March	A
	Geography	April	C

```
In [92]: df.swaplevel()
```

Out[92]:

Grade			
Final exam	January	History	A
	February	Geography	B
Coursework	March	History	A
	April	Geography	C

In [93]:

```
df.swaplevel(0,1)
```

Out[93]:

Grade			
History	Final exam	January	A
Geography	Final exam	February	B
History	Coursework	March	A
Geography	Coursework	April	C

In [94]:

```
df.swaplevel(0,2)
```

Out[94]:

Grade			
January	History	Final exam	A
February	Geography	Final exam	B
March	History	Coursework	A
April	Geography	Coursework	C

In [97]:

```
df4
df4.swaplevel()
df4.swaplevel(axis=1)
```

Out[97]:

name		A			B			C		
count		1	2	3	1	2	3	1	2	3
Year	Month									
1995	May	2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9
	Dec	-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4
2000	May	-1.7	2.0	-0.5	-0.4	-1.3	0.8	-1.6	-0.2	-0.9
	Dec	0.4	-0.5	-1.2	-0.0	0.4	0.1	0.3	-0.6	-0.4

Out[97]:

name		A			B			C		
count		1	2	3	1	2	3	1	2	3
Month	Year									
May	1995	2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9
Dec	1995	-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4
May	2000	-1.7	2.0	-0.5	-0.4	-1.3	0.8	-1.6	-0.2	-0.9
Dec	2000	0.4	-0.5	-1.2	-0.0	0.4	0.1	0.3	-0.6	-0.4

Out[97]:

	count	1	2	3	1	2	3	1	2	3
	name	A	A	A	B	B	B	C	C	C
Year	Month									
1995	May	2.3	-1.5	0.0	-0.2	1.5	1.5	0.2	0.4	-0.9
	Dec	-2.0	-0.3	0.2	1.2	1.2	-0.4	-0.3	-1.0	-1.4
2000	May	-1.7	2.0	-0.5	-0.4	-1.3	0.8	-1.6	-0.2	-0.9
	Dec	0.4	-0.5	-1.2	-0.0	0.4	0.1	0.3	-0.6	-0.4

[정리] 다중인덱스의 접근 방법

- 인덱스가 하나가 아니므로 묶어서(튜플로) 전달
- 열 접근 : df[(튜플)]
- 행 접근 : df.loc[(튜플)]
- 참고. df.iloc[]은 정수위치로 접근하여 다중인덱스에 구애받지 않음

8. 다중인덱스의 행/열 추가

In [98]:

```
data = np.round(np.random.rand(6, 4), 2)
columns = pd.MultiIndex.from_product([['A', 'B'], ['C1', 'C2']],
                                     names=['cidx1', 'cidx2'])
index = pd.MultiIndex.from_product([['M', 'F'], ['id1', 'id2', 'id3']],
                                   names=['ridx1', 'ridx2'])
df = pd.DataFrame(data=data, index=index, columns=columns)
df
```

Out[98]:

		cidx1	A		B	
		cidx2	C1	C2	C1	C2
ridx1	ridx2					
M	id1	0.30	0.12	0.32	0.41	
	id2	0.06	0.69	0.57	0.27	
	id3	0.52	0.09	0.58	0.93	
F	id1	0.32	0.67	0.13	0.72	
	id2	0.29	0.18	0.59	0.02	
	id3	0.83	0.00	0.68	0.27	

In [99]:

```
df2 = df.copy()
df2
```

Out[99]:

		cidx1		A		B	
		cidx2		C1	C2	C1	C2
ridx1	ridx2						
M	id1	0.30	0.12	0.32	0.41		
	id2	0.06	0.69	0.57	0.27		
	id3	0.52	0.09	0.58	0.93		
F	id1	0.32	0.67	0.13	0.72		
	id2	0.29	0.18	0.59	0.02		
	id3	0.83	0.00	0.68	0.27		

값입력을 위한 위치 지정 실수?

In [101...

```
df2.loc[('F','id1'),('B','C1')]
```

Out[101...

np.float64(0.13)

In [102...

```
df2.loc[('F','id1'),('B','c1')] = 100
```

In [103...

```
df2
```

Out[103...

		cidx1		A		B	
		cidx2		C1	C2	C1	C2
ridx1	ridx2						
M	id1	0.30	0.12	0.32	0.41	NaN	
	id2	0.06	0.69	0.57	0.27	NaN	
	id3	0.52	0.09	0.58	0.93	NaN	
F	id1	0.32	0.67	0.13	0.72	100.0	
	id2	0.29	0.18	0.59	0.02	NaN	
	id3	0.83	0.00	0.68	0.27	NaN	

In [104...

```
df2 = df2.drop(columns=('B', 'c1'))
df2
```

Out[104...

		cidx1		A		B	
		cidx2		C1	C2	C1	C2
ridx1	ridx2						
M	id1	0.30	0.12	0.32	0.41		
	id2	0.06	0.69	0.57	0.27		
	id3	0.52	0.09	0.58	0.93		
F	id1	0.32	0.67	0.13	0.72		
	id2	0.29	0.18	0.59	0.02		
	id3	0.83	0.00	0.68	0.27		

각 행의 총합을 마지막 열로 추가

```
In [106... df2.sum(axis=1)
df2[('Row', 'Sum')] = df2.sum(axis=1)
df2
```

Out[106... ridx1 ridx2
M id1 1.15
id2 1.59
id3 2.12
F id1 1.84
id2 1.08
id3 1.78
dtype: float64

Out[106...

		cidx1		A		B	Row
		cidx2	C1	C2	C1	C2	Sum
	ridx1	ridx2					
	M	id1	0.30	0.12	0.32	0.41	1.15
		id2	0.06	0.69	0.57	0.27	1.59
		id3	0.52	0.09	0.58	0.93	2.12
	F	id1	0.32	0.67	0.13	0.72	1.84
		id2	0.29	0.18	0.59	0.02	1.08
		id3	0.83	0.00	0.68	0.27	1.78

```
In [107... df2.sum(axis=0)
```

Out[107... cidx1 cidx2
A C1 2.32
C2 1.75
B C1 2.87
C2 2.62
Row Sum 9.56
dtype: float64

각 열의 총합을 마지막 행으로 추가

```
In [109... df2.loc[('Col', 'Sum'), :] = df2.sum(axis=0)
df2
```

Out[109...

		cidx1	A		B	Row	Sum
		cidx2	C1	C2	C1	C2	Sum
ridx1	ridx2						
M	id1	0.30	0.12	0.32	0.41	1.15	NaN
	id2	0.06	0.69	0.57	0.27	1.59	NaN
	id3	0.52	0.09	0.58	0.93	2.12	NaN
F	id1	0.32	0.67	0.13	0.72	1.84	NaN
	id2	0.29	0.18	0.59	0.02	1.08	NaN
	id3	0.83	0.00	0.68	0.27	1.78	NaN
Col		NaN	NaN	NaN	NaN	NaN	NaN
Sum		2.32	1.75	2.87	2.62	9.56	0.0

In [112...

```
df2.index
# df2.drop(index=('Col ',      ''), inplace=True)
```

Out[112...

MultiIndex([('M', 'id1'),
 ('M', 'id2'),
 ('M', 'id3'),
 ('F', 'id1'),
 ('F', 'id2'),
 ('F', 'id3'),
 ('Col', ''),
 ('Col', 'Sum')],
 names=['ridx1', 'ridx2'])

In [115...

```
df2.columns
# df2.drop(columns=('Sum',      ''), inplace=True)
# df2
```

Out[115...

MultiIndex([('A', 'C1'),
 ('A', 'C2'),
 ('B', 'C1'),
 ('B', 'C2'),
 ('Row', 'Sum'),
 ('Sum', '')],
 names=['cidx1', 'cidx2'])

Out[115...

		cidx1	A		B	Row	
		cidx2	C1	C2	C1	C2	Sum
ridx1	ridx2						
M	id1	0.30	0.12	0.32	0.41	1.15	
	id2	0.06	0.69	0.57	0.27	1.59	
	id3	0.52	0.09	0.58	0.93	2.12	
F	id1	0.32	0.67	0.13	0.72	1.84	
	id2	0.29	0.18	0.59	0.02	1.08	
	id3	0.83	0.00	0.68	0.27	1.78	
Col	Sum	2.32	1.75	2.87	2.62	9.56	

9. 다중인덱스 정렬

1) sort_index()

[형식] `sort_index(*, axis=0, level=None, ascending=True, inplace=False,)`

- 행/열 인덱스 기준으로 정렬
- 기본 정렬 방식 : 오름차순 정렬
- 내림차순 : `ascending=False` 설정

행인덱스 정렬

In [117...

```
df
```

Out[117...

		cidx1		A		B	
		cidx2	C1	C2	C1	C2	
ridx1	ridx2						
M	id1	0.30	0.12	0.32	0.41		
	id2	0.06	0.69	0.57	0.27		
	id3	0.52	0.09	0.58	0.93		
F	id1	0.32	0.67	0.13	0.72		
	id2	0.29	0.18	0.59	0.02		
	id3	0.83	0.00	0.68	0.27		

In [118...

```
df.sort_index()  
df.sort_index(ascending=False)
```

Out[118...

		cidx1		A		B	
		cidx2	C1	C2	C1	C2	
ridx1	ridx2						
F	id1	0.32	0.67	0.13	0.72		
	id2	0.29	0.18	0.59	0.02		
	id3	0.83	0.00	0.68	0.27		
M	id1	0.30	0.12	0.32	0.41		
	id2	0.06	0.69	0.57	0.27		
	id3	0.52	0.09	0.58	0.93		

Out[118...

		cidx1		A		B	
		cidx2		C1	C2	C1	C2
ridx1	ridx2						
M	id3	0.52	0.09	0.58	0.93		
	id2	0.06	0.69	0.57	0.27		
	id1	0.30	0.12	0.32	0.41		
F	id3	0.83	0.00	0.68	0.27		
	id2	0.29	0.18	0.59	0.02		
	id1	0.32	0.67	0.13	0.72		

In [119...

```
df.sort_index(level=0)
df.sort_index(level=1)
df.sort_index(level=-1, ascending=False)
```

Out[119...

		cidx1		A		B	
		cidx2		C1	C2	C1	C2
ridx1	ridx2						
F	id1	0.32	0.67	0.13	0.72		
	id2	0.29	0.18	0.59	0.02		
	id3	0.83	0.00	0.68	0.27		
M	id1	0.30	0.12	0.32	0.41		
	id2	0.06	0.69	0.57	0.27		
	id3	0.52	0.09	0.58	0.93		

Out[119...

		cidx1		A		B	
		cidx2		C1	C2	C1	C2
ridx1	ridx2						
F	id1	0.32	0.67	0.13	0.72		
M	id1	0.30	0.12	0.32	0.41		
F	id2	0.29	0.18	0.59	0.02		
M	id2	0.06	0.69	0.57	0.27		
F	id3	0.83	0.00	0.68	0.27		
M	id3	0.52	0.09	0.58	0.93		

Out[119...

cidx1		A		B	
cidx2		C1	C2	C1	C2
ridx1	ridx2				
M	id3	0.52	0.09	0.58	0.93
	F id3	0.83	0.00	0.68	0.27
	M id2	0.06	0.69	0.57	0.27
F	id2	0.29	0.18	0.59	0.02
	M id1	0.30	0.12	0.32	0.41
	F id1	0.32	0.67	0.13	0.72

열인덱스 기준으로 정렬

In [121...

```
df.sort_index(axis=1, ascending=False)
df.sort_index(axis=1, level=1)
df.sort_index(axis=1, level=0, ascending=False)
```

Out[121...

cidx1		B		A	
cidx2		C2	C1	C2	C1
ridx1	ridx2				
M	id1	0.41	0.32	0.12	0.30
	id2	0.27	0.57	0.69	0.06
	id3	0.93	0.58	0.09	0.52
F	id1	0.72	0.13	0.67	0.32
	id2	0.02	0.59	0.18	0.29
	id3	0.27	0.68	0.00	0.83

Out[121...

cidx1		A	B	A	B
cidx2		C1	C1	C2	C2
ridx1	ridx2				
M	id1	0.30	0.32	0.12	0.41
	id2	0.06	0.57	0.69	0.27
	id3	0.52	0.58	0.09	0.93
F	id1	0.32	0.13	0.67	0.72
	id2	0.29	0.59	0.18	0.02
	id3	0.83	0.68	0.00	0.27

Out[121...

		cidx1		B		A	
		cidx2		C2	C1	C2	C1
ridx1	ridx2						
M	id1	0.41	0.32	0.12	0.30		
	id2	0.27	0.57	0.69	0.06		
	id3	0.93	0.58	0.09	0.52		
F	id1	0.72	0.13	0.67	0.32		
	id2	0.02	0.59	0.18	0.29		
	id3	0.27	0.68	0.00	0.83		

2) sort_values()

[형식]

```
sort_values(by, *, axis=0, ascending=True, inplace=False, )
```

- 특정 컬럼 값을 기준으로 정렬
- by = 특정컬럼
 - 특정컬럼이 다중인덱스 일 경우 컬럼명을 튜플로 전달

df의 A.C1 컬럼을 기준으로 정렬

In [123...

```
df.sort_values(by=('A', 'C1'))
```

Out[123...

		cidx1		A		B	
		cidx2		C1	C2	C1	C2
ridx1	ridx2						
M	id2	0.06	0.69	0.57	0.27		
F	id2	0.29	0.18	0.59	0.02		
M	id1	0.30	0.12	0.32	0.41		
F	id1	0.32	0.67	0.13	0.72		
M	id3	0.52	0.09	0.58	0.93		
F	id3	0.83	0.00	0.68	0.27		

In [125...

```
# df.sort_values(by='A') # ValueError: The column label 'A' is not unique.  
# 두 개 이상의 컬럼을 갖는 레벨이므로 정렬 불가  
# For a multi-index, the label must be a tuple with elements corresponding to each level.
```

In [127...

```
# df.sort_values(by='A.C1') # .연사자 이용한 컬럼명 접근 불가
```
